

zeroTrace：轻量级用户态动态探针项目文档

2026 年全国大学生计算机系统能力大赛操作系统设计赛

OS 功能挑战赛 · proj40

参赛赛道	2026 年全国大学生计算机系统能力大赛操作系统设计赛 – OS 功能挑战赛
赛题编号	proj40
赛题名称	轻量级用户态动态探针设计与实现
项目名称	zeroTrace
队伍名称	动感光波 biubiubiu
学校	中国矿业大学
队员	舒钰尧、曹佳一、倪晨曦
指导教师	林果园

项目文档

2026 年 6 月 30 日

目录

1	引言	3
1.1	设计目标	3
2	需求对应关系与实现特点	4
2.1	赛题要求对应关系	4
2.2	同类方案对比	5
2.3	实现特点	6
2.4	创新性分析	6
3	系统架构	7
3.1	总体结构	7
3.2	模块划分	7
3.3	核心数据结构	8
3.4	生命周期流程	9
4	关键实现	10
4.1	入口 patch 与 trampoline	10
4.2	前导指令重定位	11
4.3	return probe 与返回链接管	11
4.4	trace buffer 发布协议	12
4.5	条件过滤与热更新	13
4.6	签名配置与参数解码	14
4.7	多线程 patch 安全	15
4.8	多架构后端	16
5	实验设计与结果	21
5.1	实验环境	21
5.2	正确性实验	21
5.3	性能实验方法	22
5.4	性能结果	23
5.5	多线程与热更新压力结果	24
6	使用与复现	25
6.1	依赖与运行前准备	25
6.2	构建与测试	25
6.3	CLI 示例	26

7 分工说明	27
8 AI 辅助工具使用说明	27
9 结论	27

摘要

zeroTrace 是一套面向 Linux 用户态程序的轻量级动态探针系统。传统 uprobe 在命中用户态函数时通常要切到内核侧处理事件，高频函数上的上下文切换会明显放大观测成本。zeroTrace 把安装阶段和命中阶段分开：tracer 进程负责安装、更新和清理 probe；函数命中后的快速路径留在目标进程用户态内部，由入口 patch、trampoline、payload stub 和共享 trace buffer 完成参数采集、返回值捕获和事件输出。

系统支持动态库函数追踪、前 6 个参数捕获、return probe、多 probe 共存、动态开关、线程组级 patch 安全、条件过滤、探针热更新和探针内调用目标进程函数，并提供 x86_64 与 aarch64 两套 ISA 后端。日志沿用 perf/ptrace 风格字段，记录 comm、pid/tid、CPU、CLOCK_MONOTONIC 时间戳和事件类型，便于和系统级 trace 事件放到同一条时间线上分析。本次 x86_64 评测中，zeroTrace 单次 probe 额外开销均值为 174.45 ns，比 kernel uprobe 低 11.08 倍；在 Google Cloud T2A aarch64 环境中，zeroTrace 额外开销均值为 237.02 ns，比 kernel uprobe 低 2.02 倍。两套环境中的 probe 安装/卸载延迟均低于赛题指标要求。

关键词：用户态动态探针；ptrace；trampoline；return probe；trace buffer；多线程安全；热更新

1 引言

动态追踪的价值在于“程序仍然按原来的方式运行，但开发者能看到更细的执行事实”。在调试线上服务、分析性能抖动、定位库函数调用链时，开发者往往希望临时插入 probe，观察某个函数的参数、返回值和调用频率。Linux 已经提供 uprobe、perf、ptrace 等工具，但其中用户态函数 probe 多数需要在命中时进入内核。低频函数还能接受这种成本；一旦函数被百万次调用，观测本身就on能改变被观测对象。

proj40 要求实现一种轻量级用户态动态探针。普通调试器式做法可以很快 trace 到函数，但每次命中都触发内核处理，热点函数上的成本会被放大。zeroTrace 把这部分成本放到安装阶段：先使用 ptrace 取得目标进程控制权，注入 payload 并改写函数入口；目标进程恢复运行后，函数直接跳入进程内的 trampoline 和 stub，事件写入共享 ring buffer，再由 tracer 异步读取。

本文档整理 zeroTrace 的设计、实现和实验记录。后文会对照赛题要求说明支持情况，并依次介绍总体架构、关键实现、正确性测试和性能评测。

1.1 设计目标

zeroTrace 的目标集中在运行中观测：目标程序继续运行，热点函数的观测成本压低到可接受范围。项目把复杂操作前移到安装阶段：符号解析、远程注入、trampoline 构造、线程组停止和入口 patch 都由 tracer 完成；函数命中后，目标进程内的执行路径集中在现场保存、事件写入和返回链跳转。

这个目标落到实现上，主要对应四个方面：函数命中后的处理留在目标进程内，减少热点函数反复切到内核的成本；参数、返回值和调用现场按目标 ABI 采集，函数本身仍按原来的语义执行；probe 可以在进程运行期间安装、禁用、卸载和更新；多线程场景下，patch 前后先收敛线程组，等所有线程离开入口改写区域后再动手。

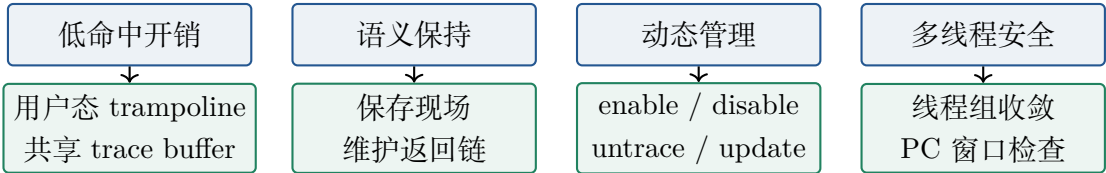


图 1: zeroTrace 的主要设计目标

2 需求对应关系与实现特点

2.1 赛题要求对应关系

表 1 和表 2 汇总了 zeroTrace 对基础功能 F1–F7 与进阶功能 A1–A5 的支持情况。

表 1: 基础功能支持情况

编号	赛题要求	zeroTrace 实现
F1	动态库函数探针插入，函数入口命中后直接跳到用户态 trampoline	支持 read/write/printf 等动态库函数。x86_64 使用 jmp qword ptr [rip+0] 模板，aarch64 使用 ldr x16; br x16 模板，命中后进入目标进程内的 trampoline。
F2	函数参数获取，至少前 6 个参数，遵循 ABI	x86_64 映射 rdi/rsi/rdx/rcx/r8/r9，aarch64 映射 x0..x5；浮点参数映射到统一 fp_args[] 槽位。
F3	函数返回值捕获	entry stub 保存真实返回地址并改写返回链；函数返回时先进入 exit stub，记录返回值后再跳回原返回地址。
F4	探针清理，恢复原始指令并释放远程资源	untrace/detach 按 probe 元数据恢复入口字节；trampoline slot 释放后进入空闲表，pool 全空时远程 munmap。清理用例会记录入口字节和 /proc/<pid>/maps 变化。
F5	探针动态开关	disable 恢复入口 patch，probe 元数据与 slot 继续可用；enable 复用原 slot 重装 patch。多线程用例会在目标运行期间反复切换。
F6	多探针共存，同一进程至少 16 个并发探针	probe 表容量为 32，自动化测试在同一进程中安装 probe_fn01..probe_fn16 并校验全部命中。
F7	多线程安全，不导致崩溃或死锁	线程组级 seize/interrupt/continue/detach，运行中刷新新线程；patch 前检查各 TID 的 PC，位于 patch 区间时 single-step 离开。

表 2: 进阶功能支持情况

编号	进阶要求	zeroTrace 实现
A1	x86_64 + ARM64/aarch64 多架构	公共控制层复用，ISA 相关 patch、远程执行、trampoline builder 和 stub 分别落在 src/isa/x86_64 与 src/isa/aarch64。
A2	条件探针	if 后字符串按完整布尔表达式解析，支持 arg0..arg5、十进制/十六进制常量、比较、算术、括号和短路求值。
A3	与 perf/fttrace 事件流合流	日志包含 comm、pid/tid、CPU、单调时钟时间戳和事件名；scripts/merge_trace_events.py 能把 zeroTrace、perf script 和 fttrace 风格事件按时间排序。
A4	探针内调用目标进程函数	update <probe> call <callee> ... 会把 callee 地址和参数来源写入远程 action 表；payload 在目标进程上下文内调用 callee 并写 call event。
A5	探针热更新	filter、call action、enable/disable 状态可在目标运行中更新；热更新用例包含 staged 更新和 live call action 切换。

2.2 同类方案对比

功能挑战赛道要求说明项目测试结果的功能、性能和创新性，并与类似项目进行对比。zeroTrace面向“用户态函数命中后的低开销观测”这一场景设计。表 3 从触发机制、参数表达能力和使用场景三个角度对比常见方案。

表 3: 同类动态追踪方案对比

方案	典型机制	优势	zeroTrace 的差异
kernel uprobe / eBPF	用户态函数命中后进入内核，事件由内核侧处理	内核生态成熟，能与 perf、fttrace、BPF map 等工具组合	命中路径会发生用户态到内核态切换；zeroTrace 把 entry/return 处理放在目标进程内，事件写入共享 trace buffer，x86_64 与 aarch64 benchmark 中额外开销均值分别为 174.45 ns/call 和 237.02 ns/call。
ltrace	依赖动态链接和 PLT 调用路径，按函数签名格式化库函数调用	使用简单，配置文件适合描述 libc 函数签名	ltrace 更偏调试输出；zeroTrace 面向运行中 patch、return probe、多线程安全和低开销 benchmark，同时借鉴 ltrace 风格配置完成参数展示。

方案	典型机制	优势	zeroTrace 的差异
SystemTap / DTrace 类工具	通过脚本化探针语言描述事件和动作	表达能力强，适合系统级观测与复杂脚本	zeroTrace 的脚本能力较轻，重点是用户态函数入口和返回链；条件过滤和 call action 固定在本项目场景内，换取更短的热路径。
perf / ftrace	采集内核事件、采样事件或 tracepoint，输出统一时间线	系统事件生态完整，适合性能分析和调度分析	zeroTrace 输出兼容 perf/ftrace 风格字段，并提供合流脚本；它补充的是普通用户态函数参数和返回值。
bpftime / 用户态 eBPF 运行时	将部分 eBPF 执行放到用户态，减少内核交互	保留 eBPF 编程模型，适合复用 BPF 生态	zeroTrace 采用 trampoline、stub 和 C payload 直接组织执行流；实现边界更窄，但便于针对赛题要求控制 ABI、返回值和线程组 patch。

2.3 实现特点

zeroTrace 的实现重点放在函数命中后的执行流、返回链、多线程控制和交互使用上。probe 命中后，执行流直接进入目标进程内的 trampoline 和 payload stub，事件记录也在目标进程内完成；tracer 在后台读取共享内存。return probe 与 entry probe 使用同一条返回链，entry 和 return 通过 call_id 配对，既能打印返回值，也能继续扩展为耗时统计。多线程场景中，zeroTrace 的控制范围是整个线程组。CLI 采用调试器式交互，用户可以在目标运行时调整过滤条件、call action 和 probe 状态。

2.4 创新性分析

zeroTrace 的主要创新集中在用户态命中路径、返回链维护、线程组安全和实验验证四个方面。

- 用户态命中路径。** 函数入口 patch 到 trampoline，命中后在目标进程内完成现场保存和事件写入，高频函数的 probe 逻辑直接沿用户态路径执行。
- return probe 返回链。** x86_64 通过栈上返回地址改写闭合 entry/return 链路，aarch64 通过 LR 链路完成同样语义；TLS shadow stack 负责按线程保存真实返回地址，entry 与 return 使用 call_id 配对。
- 线程组级 patch 安全。** trace runner 在 patch 前刷新目标线程表，纳管运行中新建线程，并检查每个 stopped TID 的 PC；处于入口 patch 窗口的线程会先 single-step 到安全位置。
- 可更新 probe 行为。** filter、enable/disable 和 call action 都能在目标运行期间更新。call action 使用非零 commit token 发布远程 action 快照，payload 读取到的是完整的旧配置、

新配置或 disabled 状态。

5. 面向提交的验证矩阵。测试覆盖 F1–F7、A1–A5、trace buffer wrap、目标退出窗口、热更新压力和多线程压力；benchmark 使用 5 轮重复实验输出 mean/min/max/stdev，并给出 kernel uprobe 对照。

3 系统架构

3.1 总体结构

zeroTrace 分为控制面和数据面。控制面运行在 tracer 进程中，负责 CLI、会话管理、ptrace 控制、远程注入、trampoline 分配、入口 patch、日志读取与格式化。数据面位于目标进程内部，由函数入口 patch、trampoline、payload stub、payload handler 和 trace buffer 组成。二者通过远程配置区、trace buffer 和 probe 元数据连接。源码导读合并在本章中展开：表 4 对应主要源文件职责，后续小节按一次 probe 的生命周期说明这些模块如何协作。

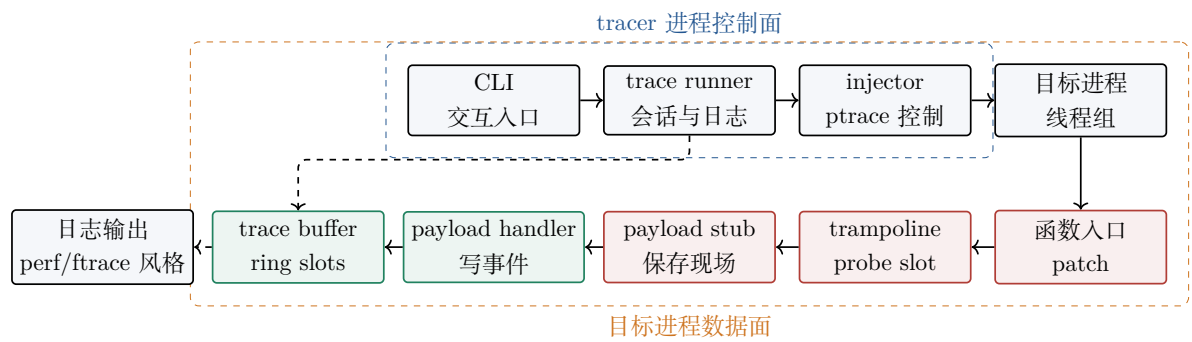


图 2: zeroTrace 控制面与数据面

图 2 中红色箭头表示函数命中后最频繁执行的部分。命中阶段主要完成时间戳记录、现场保存和事件写入，执行流留在目标进程内；tracer 的轮询读取共享 buffer，不中断目标进程运行。

3.2 模块划分

表 4: 核心模块职责

模块	层次	职责
zt_cli	交互层	解析用户命令，维护 attach 会话，触发 trace、update、disable、untrace 等操作，并在等待输入时以 50 ms 节流读取新增日志。
zt_trace_runner	控制层	串联 injector、payload、trampoline pool、filter、sigconf 和日志输出，是 active trace 的状态中心。

模块	层次	职责
zt_injector	注入层	管理 ptrace、线程组停止/恢复、新线程纳管、远程内存读写、远程 mmap/munmap、符号解析、入口 patch 和恢复。
zt_trampoline_pool	资源层	管理目标进程中的 executable trampoline pool，按 slot 分配和回收。
src/isa/<arch>	架构后端	提供远程执行、入口跳转模板、patch span 计算、trampoline builder 和 stub。
zt_payload	数据面 runtime	注入目标进程后执行，负责保存现场、维护 TLS shadow stack、写 entry/return/call event。
zt_filter	表达式层	编译和求值条件探针表达式。
zt_sigconf	展示层	根据 conf/zttrace.conf 把原始寄存器槽格式化为函数签名形式。

3.3 核心数据结构

运行态主要由 probe 表、trace buffer 和 active trace 状态组成。probe 表记录“正在追踪哪些函数”，trace buffer 承接函数命中后的事件，active trace 状态则保存 tracer 与目标进程之间共享的远程资源。

表 5: 运行态核心数据结构

结构	主要内容与作用
probe 表	保存 probe_id、符号名、模块路径、远程函数地址、trampoline 地址、原始入口字节、覆盖长度、状态和 filter。状态显式区分为 resolved/prepared/installed/disabled，便于 enable、disable、untrace 和热更新复用同一套元数据。
trace buffer	保存 probe_id、event_type、call_id、timestamp_ns、tid、cpu_id、通用参数槽、浮点参数槽和 committed_seq。entry 记录参数，return 记录返回值，call 记录 probe 内调用目标函数的结果。
call action 表	由 tracer 写入远程配置区，payload 在 entry 命中时读取。表项包含 enabled commit token、probe_id、callee 地址、参数个数和参数来源。
active trace 状态	维护 session、远程 payload 地址、trace buffer 地址、payload config 地址、trampoline pool 元数据、日志文件句柄、reader 的 last_seq 和运行状态。
entry cache	tracer 侧按 call_id 缓存 entry 信息，用于 return 配对、条件过滤同步吞掉 return，以及 read/recv/fgets 等函数在返回后读取 buffer 内容。

3.4 生命周期流程

一次 probe 从安装到清理分三步：准备运行时、安装入口跳转、消费事件。第一条 probe 会额外完成 payload 注入和共享内存初始化；后续 probe 复用同一份 runtime，再分配新的 trampoline slot。

trace runner 会记录本轮创建的远程 runtime 区域，包括 payload path、trace buffer、payload config 和 trampoline pool。删除最后一个 probe 时，清理顺序是先恢复函数入口，再释放 trampoline slot；如果 pool 已经空闲，再远程 munmap。随后释放 payload path、trace buffer 和 payload config。目标进程继续存活时，tracer 创建的匿名映射也会随之清理。

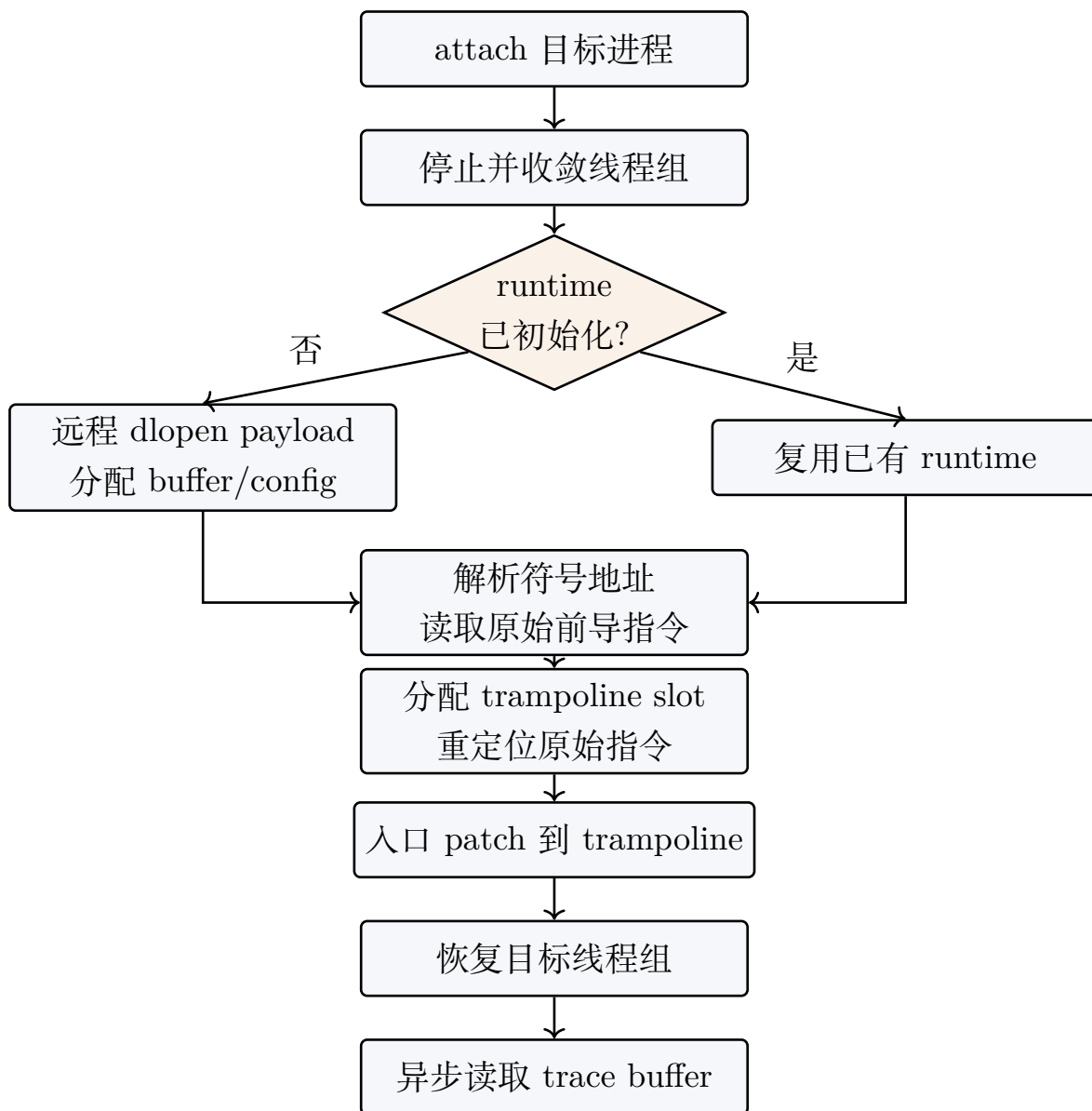


图 3: probe 安装与运行流程

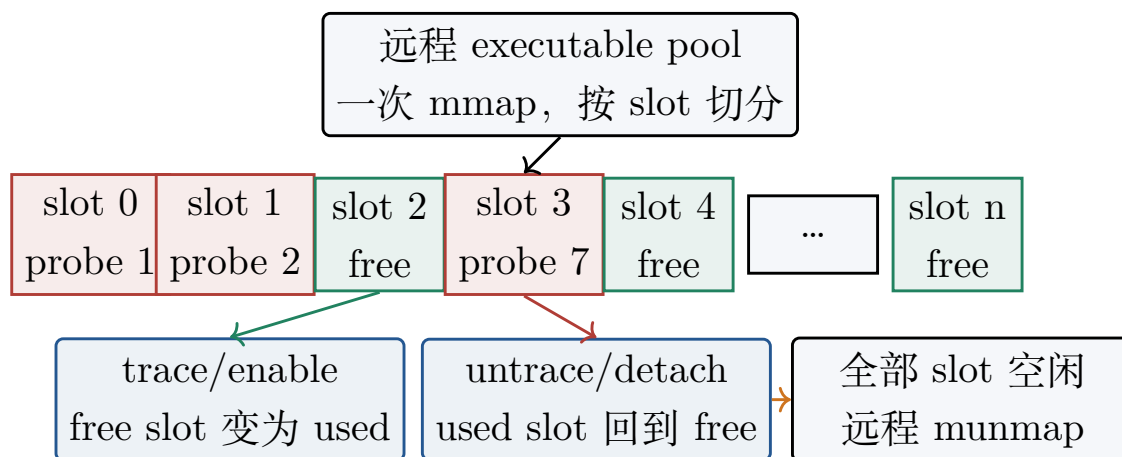


图 4: trampoline pool 与 slot 回收方式

4 关键实现

4.1 入口 patch 与 trampoline

zeroTrace 改写函数入口时写入固定长度的跳转模板。x86_64 使用 14 字节 RIP 相对间接跳转模板，模板内容为 `ff 25 00 00 00 00` 后接 64 位 trampoline 地址；aarch64 使用 16 字节 `ldr/br` 模板，并按 AAPCS64 选择 `x16/ip0` 作为跳转寄存器。两种模板都会把执行流送入远程 trampoline，再由 trampoline 负责执行被搬运的前导指令并跳回原函数。

```
x86_64 entry patch:
    jmp qword ptr [rip + 0]
    .quad trampoline_addr

aarch64 entry patch:
    ldr x16, #8
    br  x16
    .quad trampoline_addr
```

入口 patch 会覆盖目标函数前导若干条指令。x86_64 后端使用 Capstone 按指令边界累计长度，直到满足模板长度；aarch64 指令固定 4 字节，按 4 字节对齐累计。被覆盖的原始指令会复制到 trampoline 中执行，之后再跳回 `func + orig_len`。

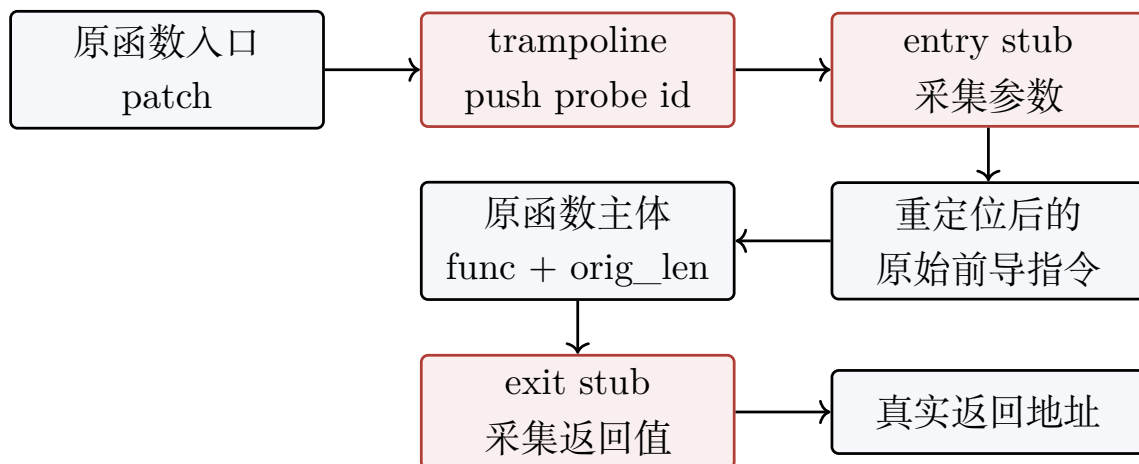


图 5: 函数命中后的控制流

4.2 前导指令重定位

trampoline builder按原函数前导语义处理指令。普通指令可以直接搬运；依赖当前位置的指令需要重定位或改写。

表 6: 前导指令重定位策略

指令类型	处理方式
普通指令	与当前 PC 无关，可原样复制。
x86_64 RIP 相对 内存访问	计算 $\text{old_target} = \text{old_next_ip} + \text{old_disp}$ ，再回填 $\text{new_disp} = \text{old_target} - \text{new_next_ip}$ 。
x86_64 call rel32	改写为绝对调用模板，消除相对偏移距离限制。
x86_64 jmp rel8/rel32	改写为绝对跳转模板。
x86_64 jcc rel8/rel32	展开为“条件反转 + 跳过绝对跳转块”的模板。
aarch64 adr/adrp	根据 trampoline 地址重新编码目标页或目标地址。
aarch64 分支类指 令	对 b/bl/b.cond/cbz/cbnz/tbz/tbnz 进行重定位；遇到暂不支持的模式时返回错误，不生成 trampoline。
aarch64 literal load	这类指令的目标地址依赖 PC；安装阶段识别后停止生成 trampoline，入口 patch 不会写入目标进程。

4.3 return probe 与返回链接管

return probe 的关键约束是保持被测函数自己的栈语义。zeroTrace 在 entry stub 中保存真实返回地址，再把函数的返回目标改成 exit stub。函数主体照常执行，ret 落到 exit stub 后记录返回寄存器，再从 TLS shadow stack 中取回真实返回地址。

x86_64 上，调用者的返回地址位于栈上，entry stub 可以直接改写对应槽位。aarch64 上，返回地址通常在 x30/lr 中，trampoline 会保存原 LR，entry stub 将 exit stub 写入返回链，真实 LR 同样进入线程本地 shadow stack。两套实现细节不同，但上层 payload 看到的事件模型一致。

x86_64 的 exit stub 入口来自原函数 ret，不同于普通 call。为了让 C handler 仍能按同一份上下文结构读取返回值，它会先补出返回地址槽和占位槽，再执行通用寄存器保存序列。记录 return event 后，exit stub 从 TLS shadow stack 弹出真实返回地址，写回预留槽位，最后通过 ret 回到原调用者。aarch64 的处理放在 x15/x30 这条 LR 链路上：x15 保存真实 LR，entry stub 返回前把 exit stub 地址带回 trampoline，trampoline 再把它写入 x30，原函数返回时进入 exit stub，最后恢复真实 LR。

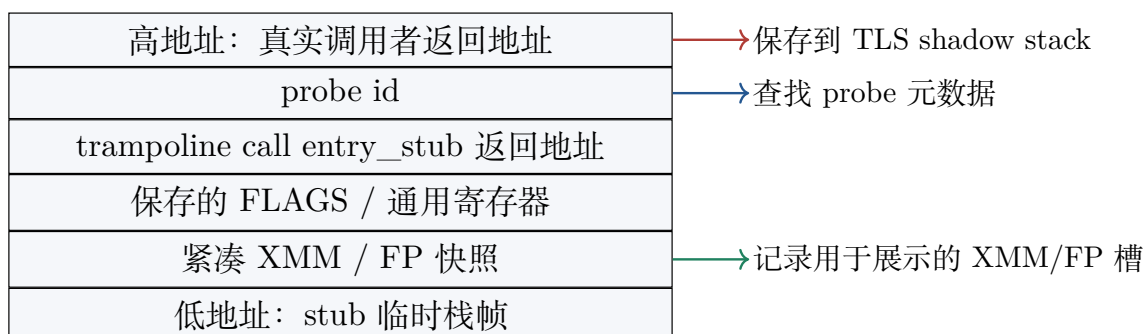


图 6: x86_64 entry stub 的逻辑栈布局

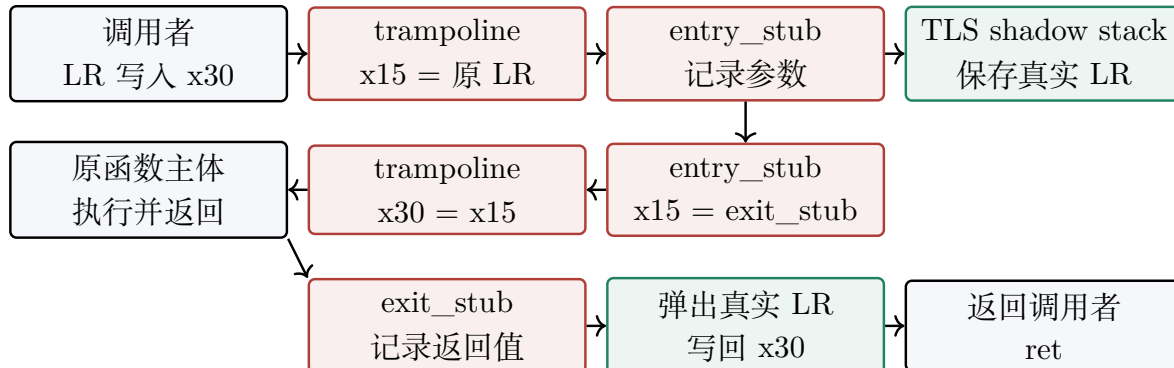


图 7: aarch64 return probe 的 LR 接管流程

浮点上下文按 ABI 参数和返回值槽位维护。x86_64 stub 对应 xmm0..xmm7，aarch64 stub 对应 FP/SIMD 参数槽位，并保留 fpcr/fpsr/nzcv 等状态字段。payload 读取统一的 fp_args[] 槽位，上层格式化逻辑只处理统一后的参数槽，使两套 ISA 后端保持一致的事件模型。

4.4 trace buffer 发布协议

payload 与 tracer 之间通过目标进程内的 ring buffer 传递事件。payload 写入时先用原子序号 reserve 一个 slot，直接在 slot 内填充 event 字段，最后写入 committed_seq。tracer 读取时先读 header；序号有变化时再按范围读取事件区，并校验 slot 前后的提交序号。

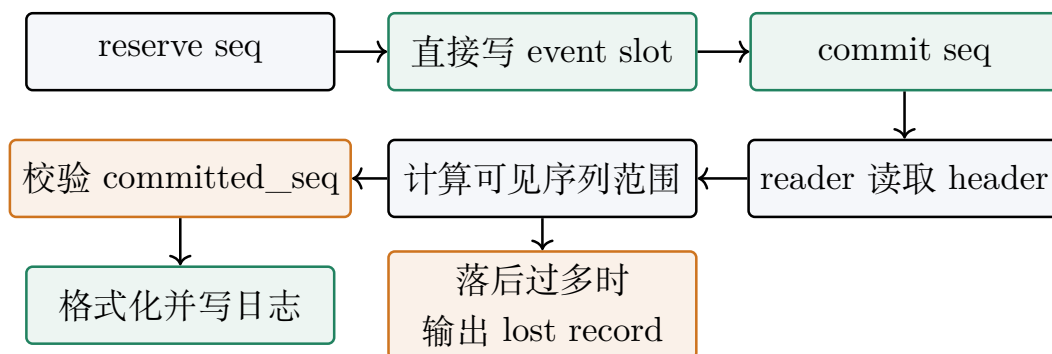


图 8: trace buffer 写入与消费协议

这套协议主要处理两种异常情况：slot 尚未提交时，reader 停在对应序列，下一轮再读，不消费半写入事件；reader 落后超过 ring capacity 时，日志中会出现 `ztrace:lost`，并写出丢失范围。

tracer 读取目标内存时优先使用 `process_vm_readv`，失败后再回退到 `ptrace` 逐字读取。正常轮询先读取 header 中的 magic 和写入序号；序号未变化时停在 header。发生 wrap 时，reader 会跳到仍可见的最早序列，并在日志中记录丢失范围。目标进程退出窗口需要单独处理：如果远程读取失败，trace runner 会结合 `kill(pid, 0)`、`/proc/<pid>/stat` 中的 zombie/dead 状态、trace buffer 映射是否仍出现在 `/proc/<pid>/maps` 中以及短暂确认窗口判断是否属于正常退出。确认退出后关闭 active trace，后续重复 poll 返回完成态，将正常退出和 trace 错误区分开。

4.5 条件过滤与热更新

条件过滤放在 tracer 消费 entry event 时求值，目标进程里的 payload 继续负责采集快照和写事件。若 entry 不满足条件，trace runner 会把该 `call_id` 标记为 suppressed；后续 return event 也按同一个标记丢弃。

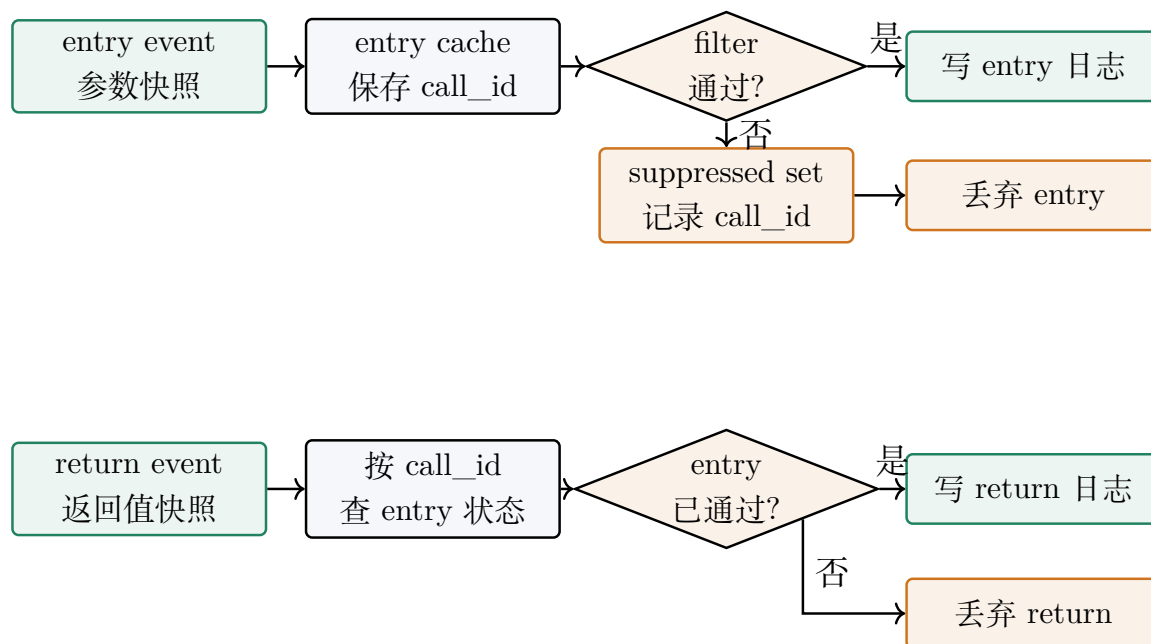


图 9: 条件过滤与 return 事件配对

```

trace write if arg0 == 1 && arg2 > 0
trace probe_fn01 if arg0 >= 0x10 && arg0 < 0x20
update probe_fn01 if arg0 >= 100
update probe_fn01 clear

```

`update` 采用替换语义。filter 更新发生在 tracer 侧，目标进程继续运行；call action 更新会写入远程 action 表，并通过非零 commit token 发布完整快照；enable/disable 涉及入口 patch，会短暂停住线程组。

filter 解析借鉴 NEMU 表达式求值思路：先把 if 后面的字符串 token 化，再按优先级递归求值。表达式支持 `arg0..arg5`、十进制和十六进制常量、比较、算术、括号、`&&/||/!`。运行期按 entry event 中的参数槽求值，`&&` 和 `||` 使用短路语义；例如 `arg0 == 0 || 10 / arg0 > 1` 左侧成立后直接返回 true，右侧除法被跳过。

call action 更新需要同步目标进程中的 action 表。tracer 写入时先把对应 slot 的 `enabled` token 清零，再写入 `probe_id`、callee 地址和参数来源，最后用新的非零 token 发布。payload 读取时先后两次读取 token，中间拷贝表项快照；只有两次 token 相同且非零时才接受该快照。这样运行中切换 callee 或清空 action 时，payload 看到的是旧配置、新配置或 disabled。action 表按实际 `probe_id` 查找匹配项，长期运行中即使出现 `probe_id % 32` 碰撞，也能区分不同 probe。

4.6 签名配置与参数解码

zeroTrace 参考 `ltrace.conf` 的思路，把常见函数签名整理成简化配置。用户补一行签名即可获得参数名、类型和返回值格式化；未配置函数则退回寄存器风格显示。

```

function_name(arg_type arg_name, arg_type arg_name, ...) -> return_type

puts(const char *s) -> int
read(int fd, buffer buf, size_t count) -> long

```

```
write(int fd, const buffer buf, size_t count) -> long
malloc(size_t size) -> void *
fp_mix(double a, double b) -> double
```

参数解码按 ABI 参数槽位进行。整数和指针参数来自通用参数槽，浮点参数来自浮点参数槽；返回值也按整数返回槽和浮点返回槽分开读取。

字符串参数会通过远程内存读取转义为可打印内容，buffer 类型会结合相邻长度参数或返回值输出部分内容。可变参数函数中，参数名为 `fmt` 的格式化字符串会触发 `varargs` 展开，格式化逻辑根据签名参数名判断。实现范围覆盖寄存器快照内的整数、指针和浮点可变参数；已经落到调用栈上的 `varargs` 需要额外的栈快照机制，本版先聚焦寄存器快照内的参数。

表 7: 签名配置类型与输出行为

类型/约定	行为
<code>int/long/size_t</code>	从通用参数槽读取并按数值输出。
<code>void * / addr</code>	从通用参数槽读取并按地址输出。
<code>char * / string</code>	读取目标进程远程字符串，非打印字符输出为 <code>\xNN</code> 。
<code>buffer</code>	结合长度参数或返回值展示读写内容片段。
<code>float/double</code>	从浮点参数槽或浮点返回槽解码。
<code>fmt 参数名</code>	读取 format string，并展开寄存器内 <code>varargs</code> 。

4.7 多线程 patch 安全

多线程安全的关键在于避开 patch 与线程执行窗口互相踩踏。zeroTrace 的线程组控制分为 attach、运行态刷新和 patch 前收敛三部分。

1. attach 时枚举 `/proc/<pid>/task`，对当前线程执行 `PTRACE_SEIZE`。
2. interrupt 阶段多轮刷新线程表，把新创建线程纳入控制，直到线程表稳定且所有 tracked TID 均 stopped。
3. 运行期间 `zt_trace_poll()` 继续消费 ptrace event，并把普通 signal-delivery-stop 的信号透传回目标线程。
4. 改写或恢复入口 patch 前，读取每个 stopped TID 的 PC；若 PC 位于任一 patch 区间，则 single-step 到安全位置。
5. patch 完成后恢复线程组，或在 detach 时逐个解除 ptrace 控制。

这套流程专门处理运行中创建线程的窗口。每轮 interrupt 后都会重新读取目标线程表，发现新 TID 就继续 seize 和 interrupt；线程表稳定、tracked TID 全部停止后，才进入 patch 阶段。single-step 避让 patch 区间时会记录该线程原本需要透传的 signal，离开 patch 区间后继续交还给目标程序。

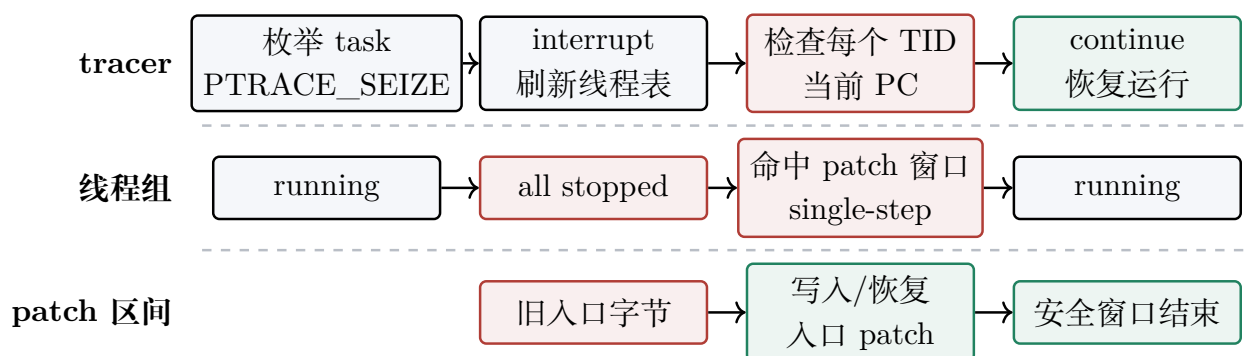


图 10: 线程组收敛与 patch 安全窗口

因此，stop、continue、enable、disable、untrace 都会覆盖整个线程组。自动化测试记录多线程 entry/return 配平、动态开关次数、线程组停止窗口和恢复后的心跳进度。

4.8 多架构后端

ptrace 注入的高层流程与 CPU 架构关系不大，但远程 syscall、远程函数调用、入口 patch 模板、trampoline 重定位和 stub 现场保存都强依赖 ISA。zeroTrace 因此把架构相关实现收敛到 `src/isa/<arch>`。

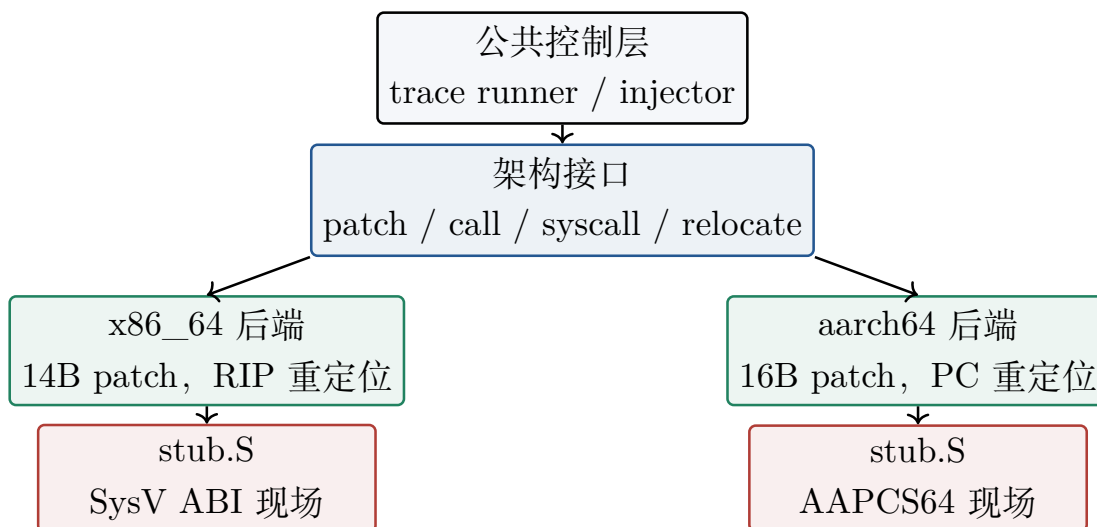


图 11: 公共控制层与 ISA 后端边界

表 8: 架构后端分工

接口/文件	职责
<code>arch.c</code>	远程 syscall、远程函数调用、patch 长度、patch span 计算和入口跳转安装。
<code>trampoline_manager.c</code>	构造该 ISA 下的 trampoline，并处理前导指令重定位。
<code>stub.S</code>	保存/恢复现场，调用 payload handler，维护 return probe 返回链。
<code>isa/common/ zt_remote_exec.c</code>	提供远程执行的通用控制流：保存现场、写入临时 stub、继续目标进程、等待陷入点、读取返回值并恢复现场；各 ISA 后端补充寄存器布局和 stub 编码。
<code>scripts/ check_arch_config.py</code>	检查 Makefile 在 ARCH=x86_64/aarch64 下是否选中正确后端和测试集合。

表 9: x86_64 与 aarch64 后端实现对照

项目	x86_64	aarch64
入口 patch	14 字节模板， <code>jmp qword ptr [rip+0]</code> 后接 64 位 trampoline 地址。	16 字节模板， <code>ldr x16, #8; br x16</code> 后接 64 位 trampoline 地址。
参数槽	SysV ABI 前 6 个整数参数来自 <code>rdi/rsi/rdx/rcx/r8/r9</code> 。	AAPCS64 前 6 个整数参数来自 <code>x0..x5</code> 。
返回值槽	整数返回值来自 <code>rax</code> ，浮点返回值来自 <code>xmm0</code> 。	整数返回值来自 <code>x0</code> ，浮点返回值来自 <code>q0/d0</code> 对应槽位。
return probe	entry stub 改写栈上的返回地址，原返回地址进入 TLS shadow stack。	trampoline 保存原 <code>x30/1r</code> ，entry stub 通过 <code>x15</code> 把 <code>exit_stub</code> 地址带回并写入 <code>x30</code> 。
远程执行	使用 <code>PTRACE_GETREGS/SETREGS</code> 操作 <code>user_regs_struct</code> 。	使用 <code>PTRACE_GETREGSET/SETREGSET</code> 操作 <code>struct user_pt_regs</code> 。
临时调用 stub	设置参数寄存器后执行目标函数，返回后用断点停回 tracer。	<code>x0/x1</code> 传参， <code>blr x16</code> 调用目标函数， <code>brk #0</code> 停回 tracer。
前导指令处理	需要处理可变长指令、RIP 相对寻址、相对 <code>call/jmp/jcc</code> 。	指令固定 4 字节，重点处理 PC 相对地址和分支立即数。

表 10: 架构特性对 zeroTrace 实现的影响

架构特性	x86_64 上的实现影响	aarch64 上的实现影响
指令长度	指令长度可变，入口 patch 前需要反汇编并累计完整指令边界。	指令固定 4 字节，patch span 按 4 字节对齐累计，边界判断更直接。
绝对跳转形式	可用 RIP 相对间接跳转，patch 模板短，但 trampoline 内要处理 RIP 相对指令。	使用 literal load 加 br x16 完成绝对跳转，模板固定为 16 字节。
返回地址位置	普通函数返回地址在调用栈上，entry stub 改写该栈槽即可接管 return probe。	返回地址在 x30/lr 中，trampoline 和 stub 需要显式传递、保存和替换 LR。
系统调用约定	系统调用号和参数寄存器与 SysV 函数 ABI 不同，远程 syscall stub 需要单独设置。	x8 保存 syscall number，x0..x5 保存参数，返回值仍在 x0。
浮点参数槽	SysV ABI 使用 xmm0..xmm7 传递常见浮点参数。	AAPCS64 使用 v0..v7/q0..q7 传递常见浮点参数。
条件码	rflags 同时承载算术状态和条件跳转依据。	nzcv 承载条件码，stub 需要配合 mrs/msr 保存恢复。

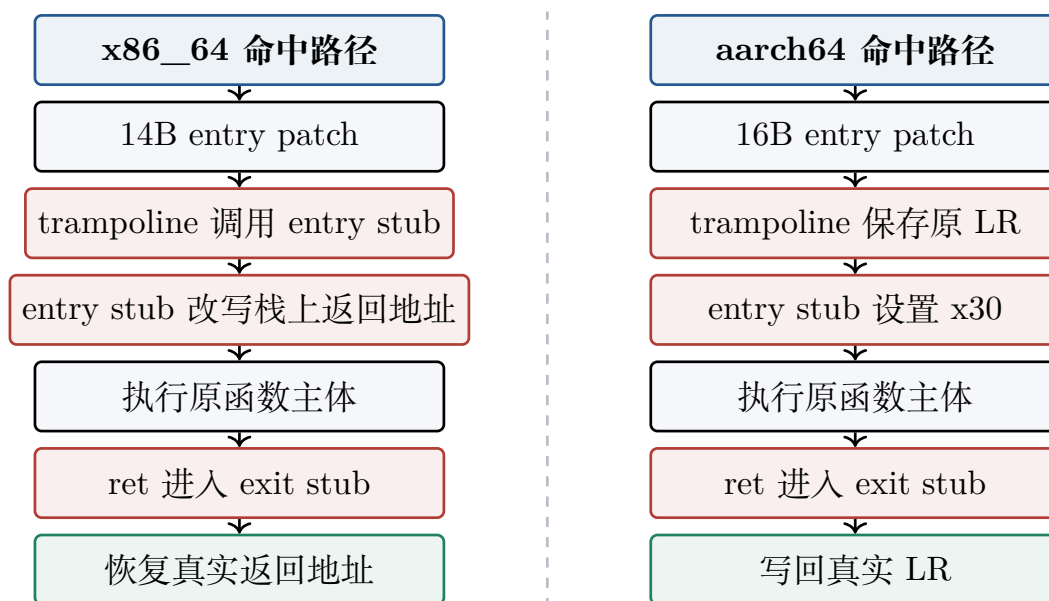


图 12: x86_64 与 aarch64 命中路径对比

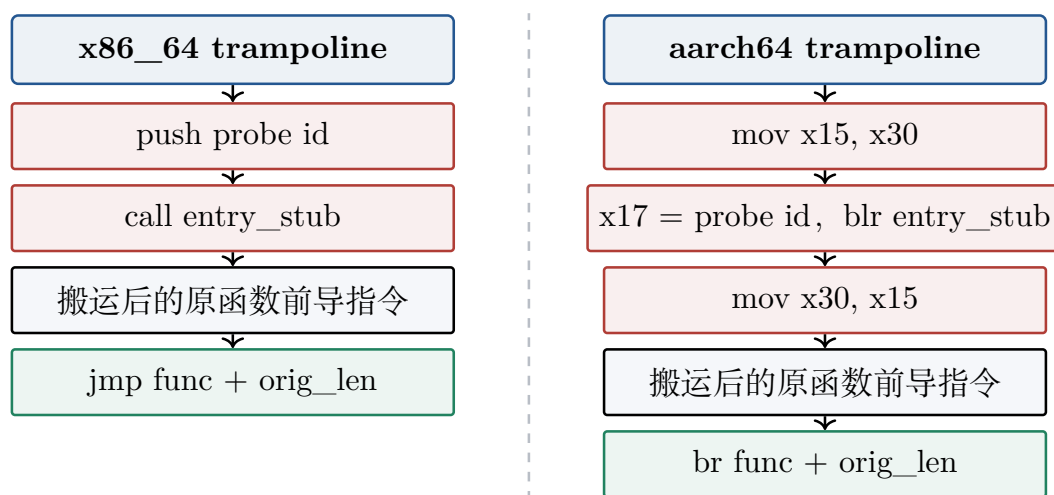


图 13: 两套 trampoline 内部组织对比

表 11: 两套 stub 的现场保存重点

现场类型	x86_64	aarch64
整数参数	保存并恢复 SysV ABI 参数寄存器, payload 读取统一 gp_arg0..5。	保存并恢复 x0..x5, payload 同样读取 gp_arg0..5。
返回值	exit stub 读取 rax 和 xmm0 对应槽位。	exit stub 读取 x0 和 q0 对应槽位。
标志位	保存/恢复 rflags, 避免条件判断受到 probe 影响。	保存/恢复 nzcv, 保持条件码语义。
浮点状态	保存用于参数展示和返回值展示的 xmm0..xmm7 槽位。	保存用于 AAPCS64 参数展示和返回值展示的 q0..q7 槽位, 并记录 fpcr/fpsr。
返回链状态	TLS shadow stack 保存原返回地址, exit stub 弹出后 ret。	TLS shadow stack 保存真实 LR, exit stub 弹出后写回 x30 并 ret。

表 12: 前导指令重定位工作量对比

重定位对象	x86_64 处理方式	aarch64 处理方式
普通指令	指令边界由 Capstone 给出, 原样复制到 trampoline。	固定 4 字节指令, 原样复制到 trampoline。
PC 相对取址	RIP 相对内存访问需要按新旧 next IP 重新计算 disp32。	adr/adrp 直接改写为等价的绝对地址装载序列。
直接调用	call rel32 改写为绝对调用模板, 避免 trampoline 与原目标距离过远。	bl imm26 改写为绝对调用模板, 目标地址由临时寄存器承载。
无条件跳转	jmp rel8/rel32 改写为绝对跳转。	b imm26 改写为绝对跳转。
条件跳转	jcc 展开为条件反转和绝对跳转块。	b.cond/cbz/cbnz/tbz/tbnz 改写为条件反转后进入绝对跳转块。
literal load	x86_64 没有同类固定模式, 主要落在 RIP 相对访存处理。	ldr literal 的目标依赖 PC, 当前安装阶段返回失败, 避免生成语义不确定的 trampoline。

aarch64 后端使用 PTRACE_GETREGSET/PTRACE_SETREGSET 读写 struct user_pt_regs, 远程 syscall 走 x0..x5/x8, 远程函数调用使用 x0/x1 传参并从 x0 读取返回值。trampoline builder 已覆盖普通指令、adr/adrp、直接分支、条件分支、cbz/cbnz 和 tbz/tbnz。遇到尚未覆盖的 PC-relative 模式时, 安装过程返回失败, tracer 保持原函数入口不变。

表 13: aarch64 后端关键约定

项目	实现方式
入口 patch	固定 16 字节模板: ldr x16, #8; br x16; .quad trampoline_addr。
远程 syscall	临时写入 svc #0; brk #0, 参数放入 x0..x5, 系统调用号放入 x8, 返回值从 x0 读取。
远程函数调用	临时 stub 使用 ldr x16, #16; blr x16; brk #0, 当前用于 x0/x1 两个参数的目标进程内函数调用。
return probe	trampoline 先把原 x30/lr 放入 x15; entry stub 记录真实 LR 后把 exit_stub 地址通过 x15 带回; trampoline 再写入 x30, 函数返回时进入 exit stub。
参数与返回值	整数参数映射到 x0..x5, 整数返回值映射到 x0; FP/SIMD 参数槽映射到 q0..q7 的低位快照, 格式化层按统一 fp_args[] 读取。
状态字段	stub 记录并恢复 nzcv, 同时保存 fpcr/fpsr, 使整数条件码和浮点控制状态不会被 payload 调用路径改变。

5 实验设计与结果

5.1 实验环境

本节数据来自 x86_64 开发机和 Google Cloud T2A aarch64 云服务器上的 benchmark 报告。两组实验均通过 Makefile 的 benchmark 目标运行，脚本检测到 tracingfs 中的 uprobe 配置入口，因此包含 kernel uprobe 对照数据。ARM 测试机为 Google Cloud t2a-standard-4，实例区域 asia-southeast1-b，CPU 为 4 核 ARM Neoverse-N1，系统为 Ubuntu 24.04.4 LTS，内核版本为 6.17.0-1020-gcp。

表 14: 实验环境

项目	x86_64 开发机	aarch64 云服务器
机器规格	13th Gen Intel(R) Core(TM) i5-13400, 16 logical CPUs	Google Cloud t2a-standard-4, 4 vCPU, 16 GiB RAM
CPU/架构	Intel x86_64	ARM Neoverse-N1, aarch64
操作系统	Linux 5.10.240-zeroVisor	Ubuntu 24.04.4 LTS, Linux 6.17.0-1020-gcp
实例区域	本地开发机	Google Cloud asia-southeast1-b
编译器	系统默认 GCC	GCC 13.3.0
编译入口	make	make ARCH=aarch64
性能入口	make benchmark	make benchmark
LaTeX 编译入口	make paper	make paper

5.2 正确性实验

正确性实验按赛题要求拆分。测试除了检查退出码，也记录关键状态变化，例如 entry/return 配平、条件过滤计数、热更新阶段范围、trampoline slot 复用和清理后的 maps 差分。

表 15: 自动化测试记录

测试	状态	关键检查项
test_libc_trace	已通过	动态库函数、参数、返回值、字符串/buffer、format string 展开。
test_context_integrity	已通过	通用寄存器、标志寄存器、浮点参数和浮点返回值上下文完整性。
test_probe_lifecycle	已通过	16 probe 共存、条件过滤、call action、slot 分配与清理恢复。

测试	状态	关键检查项
test_thread_safety	已通过	多线程并发命中、entry/return 配平、运行中 enable/disable。
test_thread_group_control	已通过	新线程纳入 ptrace 控制、线程组 stop/continue 收敛。
test_probe_hot_update	已通过	filter、call action、enable/disable 热更新。
test_poll_exit_race	已通过	短生命周期进程退出窗口，poll 正常收尾。
test_trace_buffer_reader	已通过	ring buffer wrap、lost event、未提交 slot 处理。
scripts/ check_arch_config.py	已通过	x86_64/aarch64 Makefile 后端选择检查。
scripts/ merge_trace_events.py --self-test	已通过	zeroTrace 与 perf/ftrace 风格日志排序合流。
scan-build	已通过	scan-build --status-bugs make clean all 已记录 No bugs found。

测试源码按职责拆分。自动化 runner 放在 `cases`，被 trace 的目标程序放在 `targets`；benchmark、手动 demo 和共享工具也分别放在独立目录。基础生命周期、热更新、线程组控制和多线程运行时行为都有对应用例。职责拆开以后，失败日志更容易定位到具体机制。

测试记录覆盖了 F1-F7；A1 中的 x86_64 后端已经跑完整套测试，aarch64 后端已有构建选择检查、trampoline builder 自测和 Google Cloud T2A 上的 benchmark 运行记录。

资源清理和退出窗口有单独压力记录。probe lifecycle 的 cleanup 子测试会在目标仍存活时记录 trampoline 地址、入口原始字节和 runtime mmap 区域；删除最后一个 probe 后，再检查进程 maps 和入口字节是否恢复。

目标退出窗口测试覆盖短生命周期进程退出、远程读取失败和 trace buffer 映射消失之间的竞态，已记录 1000 轮压力通过。trace buffer reader 测试会故意延迟首次 poll，让事件数量超过 ring buffer 容量，要求日志显式出现 `ztrace:lost`，把被覆盖事件的范围写出来。

5.3 性能实验方法

性能实验包含四项：baseline、zeroTrace、uprobe 和生命周期延迟。baseline 直接循环调用 `bench_getpid()`；zeroTrace 在同一目标函数上安装 probe 后执行相同循环；uprobe 在权限满足时使用 `bpftime` 对同一函数采样；生命周期延迟对同一个 probe 执行 1000 轮 install/uninstall。

benchmark 目标函数是一个 `noinline` 的 `getpid` 包装函数，内部直接发起 `getpid` syscall。这样可以避开 `libc/vDSO` 的实现差异，让 baseline、kernel uprobe 和 zeroTrace 三组实验尽量测量同一个用户态函数入口。

默认参数为 1,000,000 次调用、5 轮重复，脚本输出 mean/min/max/stddev。kernel uprobe 对照依赖 `bpftime` 与 `tracingfs` 权限；权限满足时纳入报告，权限不足时脚本会写出说明并跳

过该项。

核心指标定义为：

$$\begin{aligned} \text{overhead_per_call} &= \frac{\text{ztrace_total_ns} - \text{baseline_total_ns}}{\text{iterations}}, \\ \text{install_latency} &= \frac{\sum_{i=1}^N \text{install_ns}_i}{N}, \quad \text{uninstall_latency} = \frac{\sum_{i=1}^N \text{uninstall_ns}_i}{N}. \end{aligned} \quad (1)$$

5.4 性能结果

表 16: x86_64 与 aarch64 benchmark 结果

指标	x86_64	aarch64 / T2A
iterations	1,000,000	1,000,000
repeats	5	5
baseline total ns mean	61,239,703 ns	173,445,186 ns
baseline per call mean	61.24 ns	173.45 ns
baseline per call min/max	60.59 ns / 62.09 ns	173.11 ns / 173.67 ns
baseline per call stdev	0.66 ns	0.28 ns
uprobe total ns mean	1,994,438,922 ns	653,140,257 ns
uprobe per call mean	1,994.44 ns	653.14 ns
uprobe per call min/max	1,905.74 ns / 2,052.76 ns	650.77 ns / 657.08 ns
uprobe per call stdev	58.88 ns	2.53 ns
uprobe overhead/call mean	1,933.20 ns	479.70 ns
uprobe overhead/call min/max	1,844.99 ns / 1,990.67 ns	477.11 ns / 483.97 ns
uprobe overhead/call stdev	58.33 ns	2.67 ns
ztrace total ns mean	235,684,856 ns	410,467,298 ns
ztrace per call mean	235.68 ns	410.47 ns
ztrace per call min/max	233.38 ns / 239.31 ns	391.45 ns / 420.28 ns
ztrace per call stdev	2.55 ns	11.18 ns
ztrace overhead/call mean	174.45 ns	237.02 ns
ztrace overhead/call min/max	172.63 ns / 177.53 ns	217.84 ns / 246.61 ns
ztrace overhead/call stdev	2.28 ns	11.25 ns
ztrace vs uprobe	11.08x lower overhead	2.02x lower overhead
install latency avg	370,539 ns, 约 0.371 ms	599,175 ns, 约 0.599 ms
uninstall latency avg	83,892 ns, 约 0.084 ms	215,881 ns, 约 0.216 ms

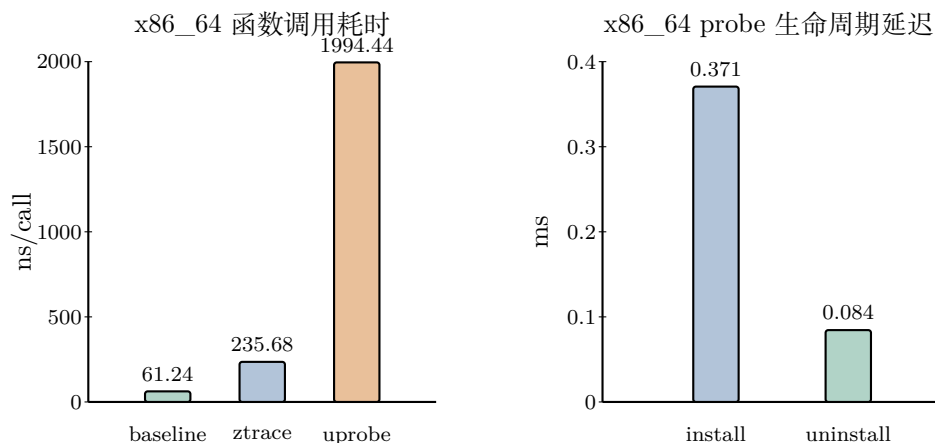


图 14: x86_64 benchmark 柱状图：函数调用开销与 probe 生命周期延迟

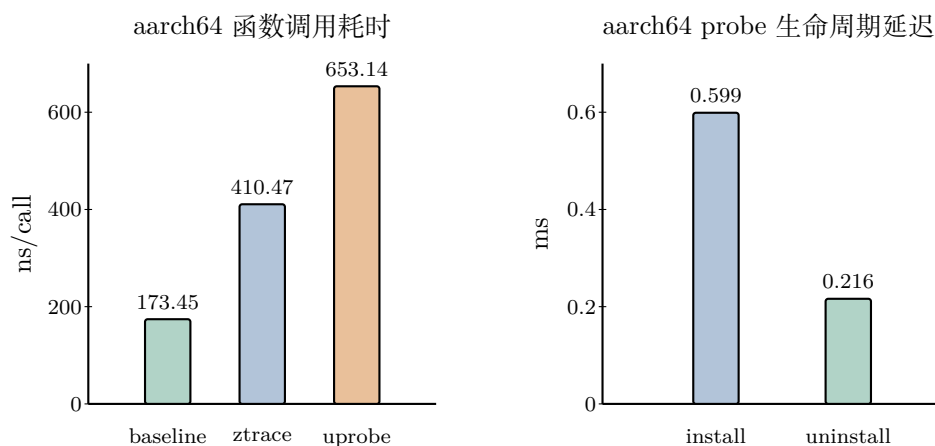


图 15: Google Cloud T2A aarch64 benchmark 柱状图：函数调用开销与 probe 生命周期延迟

在 x86_64 结果中，zeroTrace 额外开销约 174.45 ns/call，低于赛题 1000 ns 指标，也低于项目内部设定的 200 ns 优化目标。Google Cloud T2A aarch64 结果中，zeroTrace 额外开销约 237.02 ns/call，kernel uprobe 额外开销均值约 479.70 ns/call，zeroTrace 为 2.02x lower overhead。两套环境中的 install/uninstall 延迟均低于 1 ms，距离赛题 10 ms 指标还有较大余量。

5.5 多线程与热更新压力结果

多线程测试记录 entry/return 配平、线程纳管和运行中切换情况。已记录的 10 轮压力摘要如下：

```
thread_safety: 10/10 passed
  tids=12 tracked=13 toggles=12 in every round
  thread_stable strict pairs: 2338..2400 per round
  thread_pair strict pairs : 471..480 per round
  thread_mix high-frequency : >=21039 entry/return events per side

thread_group_control: 10/10 passed
```

```
rounds=80 resume_rounds=80 in every round
task_max/tracked_max range: 31..32, matched in every round
```

热更新测试检查 filter A/B/final、disable 阶段零泄漏、call action A/B 切换和 live call action 更新。已记录 100 轮压力结果：

```
test_probe_hot_update: 100/100 passed
last live call action round: hot_call_a=170 hot_call_b=45
```

这些压力结果对应并发命中、运行中状态切换、返回链配平和 call action 半写入窗口，比单次手动演示更接近真实使用场景。A4 的测试还覆盖了带参 call action：测试把 entry 参数和常量一起传给目标进程内的 callee，并检查不同参数下的返回值。slot 碰撞子测试会保留一个旧 probe，同时反复 trace/untrace 推进新 probe id，覆盖两个 probe 映射到同一 action 槽位的场景，要求两者的 call action 同时保持正确。

6 使用与复现

6.1 依赖与运行前准备

项目依赖 libcapstone、libdl 和 libreadline。在 Debian/Ubuntu 环境中可以使用以下命令安装构建依赖：

```
sudo apt install build-essential libcapstone-dev libreadline-dev
```

zeroTrace 依赖 ptrace 控制目标进程。现代 Linux 发行版通常通过 Yama 限制 ptrace 附加其他进程，实验前需要确认 `/proc/sys/kernel/yama/ptrace_scope`。若系统策略允许，可临时设置为 0：

```
cat /proc/sys/kernel/yama/ptrace_scope
echo 0 | sudo tee /proc/sys/kernel/yama/ptrace_scope
```

6.2 构建与测试

项目提供统一 Makefile。常规构建、测试和 benchmark 命令如下：

```
make
make test
make benchmark
```

Makefile 会根据构建机器选择架构后端，也可以显式指定：

```
make ARCH=x86_64
make ARCH=aarch64
make ARCH=aarch64 CC=aarch64-linux-gnu-gcc
```

LaTeX 文档可以本地编译：

```
make paper
make clean-paper
```

`paper` 目标使用 `latexmk -xelatex`, 文档使用 `ctexart` 的 Fandol 字体集, Linux 环境可以直接编译。

6.3 CLI 示例

zeroTrace 的交互方式接近调试器。用户先 `attach` 目标进程, 再安装、更新或卸载 `probe`。

```
./bin/ztrace
attach <pid>
trace add_loop
trace write if arg0 == 1 && arg2 > 0
update add_loop if arg0 >= 100
update add_loop call call_marker_args arg0 0x10 arg0
disable add_loop
enable add_loop
untrace add_loop
detach
```

日志默认写入工作目录下的 `ztrace.<pid>.log`。如果 `conf/ztrace.conf` 中存在函数签名, 日志会优先输出接近函数调用形式的内容, 例如:

```
test_loop-532386/532386 [010] 58915.513461172:
  ztrace:entry: add_loop(24, 25)
test_loop-532386/532386 [010] 58915.513468205:
  ztrace:return: add_loop -> 49
test_loop-532386/532386 [010] 58915.513461172:
  ztrace:entry: fp_add_loop(24.25, 1.5)
test_loop-532386/532386 [010] 58915.513468205:
  ztrace:return: fp_add_loop -> 25.75
```

7 分工说明

表 17: 队员分工

成员	主要工作
舒钰尧	主要负责 trace runner 与探针生命周期管理，完成 attach、trace、untrace、enable、disable、update 等上层流程串联；参与 return probe 调用配对设计，整理 entry event、return event 与 call event 的状态关系；负责部分自动化测试用例编写与回归验证。
曹佳一	主要负责 injector 与目标进程控制相关工作，包括 ptrace 线程组控制、远程内存读写、远程 mmap/munmap、payload 注入、入口 patch 与原始指令恢复；重点处理多线程目标进程中的线程枚举、停止、继续和 patch 安全窗口。
倪晨曦	主要负责 trampoline、stub 与 ISA 后端实现，完成 x86_64/aarch64 入口跳转模板、前导指令保存与跳回逻辑；参与 payload 侧现场保存、参数/返回值捕获、条件探针和 call action 联调。

8 AI 辅助工具使用说明

开发和文档整理过程中使用过 AI 辅助工具，主要用于讨论改进方案、辅助编写重复性或规则明确的代码、整理测试清单和调整文档表述。项目架构、trampoline/stub/payload 机制、线程组控制策略、性能优化方向和最终取舍由团队完成；报告中的代码、测试和性能结论以仓库内容和本地运行输出为准。

表 18: AI 辅助工具使用说明

字段	说明
工具名称	OpenAI ChatGPT / Codex 类代码助手
模型名称	OpenAI GPT-5
使用范围	改进方案讨论、重复性代码辅助、边界测试建议、benchmark 现象分析和文档措辞整理
人工完成部分	总体架构、关键机制设计、核心实现取舍、实验设计和最终结果解释
人工校验	diff 审阅、构建、定向测试、全量测试、benchmark 和文档事实核对
最终责任	团队负责方案确认、代码审阅、测试验证和最终提交

9 结论

zeroTrace 已经打通从安装 probe 到输出日志的完整流程：tracer 负责安装和管理，目标进程内部完成命中后的采集逻辑，事件通过共享 trace buffer 异步传出。项目实现了基础功能 F1-F7，并补充了条件探针、热更新、探针内调用、perf/fttrace 风格日志和多架构后端。x86_64 benchmark

显示, 单次 probe 额外开销保持在 200 ns 以内; Google Cloud T2A aarch64 benchmark 中, zeroTrace 额外开销均值为 237.02 ns/call, 相对 kernel uprobe 为 2.02x lower overhead。两套环境中的安装/卸载延迟均远低于 10 ms。

参考文献

- [1] Linux manual page: ptrace(2).
- [2] Linux manual page: process_vm_readv(2).
- [3] Linux perf documentation: perf-script.
- [4] ltrace configuration manual: ltrace.conf(5).
- [5] NJU ProjectN, NEMU expression evaluator.